

Summer school ISIMA - OTH Phase 2

Mathematical Modeling and Optimization with Julia

Prof. Dr. Viet Hung Nguyen - ISIMA - Clermont Auvergne INP

Lesson 6 - Integer Programming and Branch and Cut Algorithm

August 2026

Linear programming versus integer programming

A linear program has continuous variables:

$$\max\{c^\top x : Ax \leq b, x \in \mathbb{R}^n\}.$$

An integer program restricts some or all variables to integer values:

$$\max\{c^\top x : Ax \leq b, x \in \mathbb{Z}^n\}.$$

A mixed-integer linear program allows both continuous and integer variables:

$$x = (y, z), \quad y \in \mathbb{R}^p, \quad z \in \mathbb{Z}^q.$$

The integrality restriction changes both the geometry and the algorithmic difficulty.

A small integer program in $\mathbb{R}^2$

Consider

$$\max \quad 3x_1 + 2x_2$$

subject to

$$2x_1 + x_2 \leq 4, \quad x_1 + 2x_2 \leq 4,$$

$$x_1, x_2 \geq 0, \quad x_1, x_2 \in \mathbb{Z}.$$

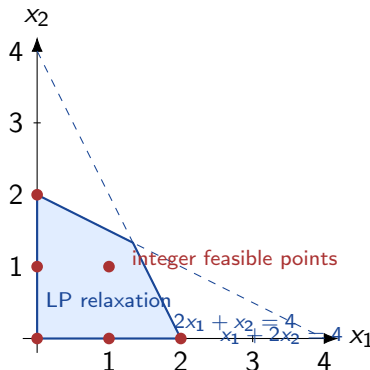
The LP relaxation is obtained by replacing

$$x_1, x_2 \in \mathbb{Z}$$

by

$$x_1, x_2 \in \mathbb{R}.$$

LP relaxation and feasible integer points



The blue polygon contains infinitely many fractional points, but only finitely many integer feasible points.

The integer hull

Let

$$P = \{x \in \mathbb{R}^2 : 2x_1 + x_2 \leq 4, x_1 + 2x_2 \leq 4, x \geq 0\}.$$

The feasible integer set is

$$X = P \cap \mathbb{Z}^2.$$

The integer hull is

$$P_I = \text{conv}(X).$$

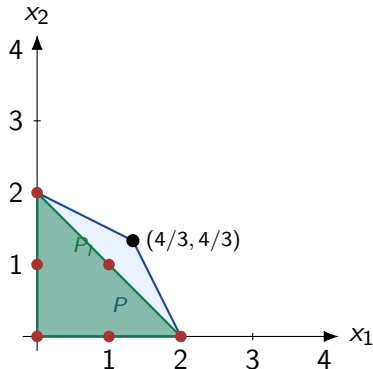
For the example,

$$X = \{(0, 0), (1, 0), (2, 0), (0, 1), (1, 1), (0, 2)\}.$$

Hence

$$P_I = \text{conv}\{(0, 0), (2, 0), (1, 1), (0, 2)\}.$$

LP relaxation strictly contains the integer hull



$$P_I \subsetneq P.$$

The fractional vertex $(4/3, 4/3)$ belongs to P , but not to P_I .

Why the integer hull matters

Optimizing a linear function over the integer feasible set is equivalent to optimizing it over the integer hull:

$$\max\{c^\top x : x \in X\} = \max\{c^\top x : x \in \text{conv}(X)\}.$$

Thus, if a linear description of P_I is available, the integer program can be solved as an LP.

For the example,

$$P_I = \{x \in \mathbb{R}_+^2 : x_1 + x_2 \leq 2\}.$$

The inequality $x_1 + x_2 \leq 2$ cuts off the fractional LP vertex.

A valid inequality

An inequality

$$\alpha^T x \leq \beta$$

is valid for X if

$$\alpha^T x \leq \beta \quad \forall x \in X.$$

For the example,

$$x_1 + x_2 \leq 2$$

is valid for all integer feasible points.

It is not valid for the LP relaxation P , because

$$(4/3, 4/3) \in P, \quad 4/3 + 4/3 = 8/3 > 2.$$

Valid inequalities strengthen LP relaxations.

A first Chvatal–Gomory cut in $\mathbb{R}^2$

The inequality $x_1 + x_2 \leq 2$ can be obtained by a generic rounding argument.

Take one third of each constraint:

$$\frac{1}{3}(2x_1 + x_2) + \frac{1}{3}(x_1 + 2x_2) \leq \frac{1}{3}4 + \frac{1}{3}4.$$

Hence

$$x_1 + x_2 \leq \frac{8}{3}.$$

Since $x_1, x_2 \in \mathbb{Z}$, the left-hand side is integer. Therefore

$$x_1 + x_2 \leq \left\lfloor \frac{8}{3} \right\rfloor = 2.$$

This is a Chvatal–Gomory cut: a nonnegative linear combination of valid inequalities, followed by rounding. It cuts the fractional vertex $(4/3, 4/3)$.

From relaxation to cutting planes

Start from an LP relaxation P .

If its optimal solution x^* is fractional, search for a valid inequality

$$\alpha^\top x \leq \beta$$

such that

$$\alpha^\top x^* > \beta.$$

Then solve the strengthened relaxation

$$P \cap \{x : \alpha^\top x \leq \beta\}.$$

This is the basic cutting-plane idea:

solve LP $\rightarrow$ separate cut $\rightarrow$ solve stronger LP.

Polyhedral combinatorics studies convex hulls of feasible solutions of discrete optimization problems.

$$\text{conv}\{x \in \{0, 1\}^n : x \text{ satisfies a combinatorial property}\}.$$

Problem	Polytope
Knapsack	knapsack polytope
Matching	matching polytope
Stable set	stable-set polytope
Traveling salesman	TSP polytope
Cut problems	cut polytope
Network design	network-design polytopes

Main questions in polyhedral combinatorics

For a discrete optimization problem, one asks:

- ① What are strong valid inequalities?
- ② Which inequalities define facets of the integer hull?
- ③ Can violated inequalities be separated efficiently?
- ④ Does the polytope admit a compact extended formulation?
- ⑤ How can these inequalities be used in Branch-and-Cut?

The goal is to replace weak LP relaxations by stronger ones.

A valid inequality

$$\alpha^\top x \leq \beta$$

defines a face of P_I :

$$F = \{x \in P_I : \alpha^\top x = \beta\}.$$

If

$$\dim(F) = \dim(P_I) - 1,$$

then the inequality defines a facet.

Facet-defining inequalities are the strongest possible linear inequalities of a given support.

In the small example, $x_1 + x_2 \leq 2$ defines a facet of P_I .

Branch-and-Bound

When cutting planes alone do not give an integer solution, the standard framework is Branch-and-Bound.

At a node N , solve a relaxation and compute a bound.

For a maximization problem:

LB = best integer solution found so far,

UB(N) = relaxation value at node N .

A node is fathomed if it is infeasible, integer, or satisfies

$$UB(N) \leq LB.$$

Branch-and-Cut combines branching and cutting planes.

1. Select a B&B node.
2. Solve its relaxation.
3. Separate violated valid inequalities.
4. Add cuts and re-solve.
5. Use the strengthened bound to fathom or branch.

The relaxation at each node is strengthened dynamically.

Modern mixed-integer solvers use many classes of cuts inside a Branch-and-Cut framework.

JuMP model for the LP relaxation

```
using JuMP
using HiGHS

model = Model(HiGHS.Optimizer)

@variable(model, x[1:2] >= 0)

@constraint(model, 2x[1] + x[2] <= 4)
@constraint(model, x[1] + 2x[2] <= 4)

@objective(model, Max, 3x[1] + 2x[2])

optimize!(model)
```

This solves the LP relaxation.

JuMP integer model

```
using JuMP
using HiGHS

model = Model(HiGHS.Optimizer)

@variable(model, x[1:2] >= 0, Int)

@constraint(model, 2x[1] + x[2] <= 4)
@constraint(model, x[1] + 2x[2] <= 4)

@objective(model, Max, 3x[1] + 2x[2])

optimize!(model)
```

The solver internally uses integer-programming algorithms, typically including Branch-and-Bound and cutting planes.

Takeaway

Integer programming replaces a continuous feasible region by a discrete feasible set.

The central geometric object is the integer hull:

$$P_I = \text{conv}(P \cap \mathbb{Z}^n).$$

A weak relaxation may contain fractional vertices that are irrelevant for the integer problem.

Polyhedral combinatorics aims to understand and strengthen this relaxation through valid inequalities.

Branch-and-Cut uses these inequalities algorithmically inside a search tree.

$$\max \quad 7x_1 + 10x_2 + 4x_3 + 4x_4 + 8x_5$$

$$3x_1 + 5x_2 + 12x_3 + 2x_4 + 6x_5 \leq 15, \quad x_i \in \{0, 1\}.$$

item i	1	2	3	4	5
profit p_i	7	10	4	4	8
weight w_i	3	5	12	2	6

The same instance is used first for cutting planes, then for Branch-and-Cut.

Root LP relaxation

The binary conditions are relaxed:

$$0 \leq x_i \leq 1.$$

The LP relaxation gives

$$x_1 = x_2 = x_4 = 1, \quad x_3 = 0, \quad x_5 = \frac{5}{6}.$$

$$z_{LP} = 7 + 10 + 4 + 8 \cdot \frac{5}{6} = 27.67.$$

This point is not feasible for the binary model.

$$x_5 = \frac{5}{6} \notin \{0, 1\}.$$

The value 27.67 is nevertheless a valid upper bound.

Cutting-plane loop

A cutting-plane algorithm strengthens the relaxation without branching.

1. Solve the current LP relaxation.
2. If the solution is integer, stop.
3. Find a violated valid inequality.
4. Add it to the LP relaxation.
5. Re-solve.

A valid inequality removes fractional points but keeps all feasible integer points.

$$P_{\text{IP}} \subseteq P_{\text{LP}} \cap \{\text{new cuts}\}.$$

Generic cuts and combinatorial cuts

Both families are valid inequalities, but they are found from different sources.

	Generic integrality cuts	Combinatorial cuts
Source	Algebraic form of the relaxation and integrality of variables	Meaning of the feasible solutions
Typical mechanism	Combine inequalities, then round coefficients or right-hand sides	Identify a forbidden or limiting combinatorial pattern
Examples	Chvatal–Gomory cuts, Gomory fractional cuts, mixed-integer rounding cuts	Knapsack cover cuts, extended covers, clique cuts for stable sets, odd-set cuts for matching, subtour cuts for TSP

A generic cut may be generated without recognizing the problem as a knapsack, matching, or routing problem. A combinatorial cut uses this interpretation explicitly.

Generic integrality cuts: Chvatal–Gomory idea

Let

$$Ax \leq b, \quad x \in \mathbb{Z}_+^n.$$

For any nonnegative multiplier vector u such that $u^\top A$ is integer,

$$u^\top Ax \leq u^\top b$$

implies

$$u^\top Ax \leq \lfloor u^\top b \rfloor.$$

More generally, by combining original inequalities and variable bounds, one obtains valid inequalities based only on integrality.

linear combination + rounding = CG cut.

A CG cut for the knapsack

Start from the capacity constraint divided by 6:

$$\frac{1}{2}x_1 + \frac{5}{6}x_2 + 2x_3 + \frac{1}{3}x_4 + x_5 \leq \frac{15}{6}.$$

Add upper-bound inequalities:

$$\frac{1}{2}x_1 \leq \frac{1}{2}, \quad \frac{1}{6}x_2 \leq \frac{1}{6}, \quad \frac{2}{3}x_4 \leq \frac{2}{3}.$$

This gives

$$x_1 + x_2 + 2x_3 + x_4 + x_5 \leq \frac{23}{6}.$$

Since the left-hand side is integer,

$$x_1 + x_2 + 2x_3 + x_4 + x_5 \leq 3.$$

The CG cut separates the root solution

At the root LP solution,

$$x = (1, 1, 0, 1, 5/6).$$

For the CG cut,

$$x_1 + x_2 + 2x_3 + x_4 + x_5 = 1 + 1 + 0 + 1 + \frac{5}{6} = \frac{23}{6} > 3.$$

Thus the fractional LP optimum is cut off.

After adding the cut, the strengthened relaxation has the integer optimum

$$x^* = (1, 1, 0, 0, 1), \quad z = 25.$$

In other words, for this objective, the root LP bound after the CG cut is already the value obtained by optimizing over the integer hull.

CG cut and integer-hull value

Let

$$P_I = \text{conv}\{x \in \{0, 1\}^5 : 3x_1 + 5x_2 + 12x_3 + 2x_4 + 6x_5 \leq 15\}.$$

Let P^{CG} be the LP relaxation strengthened by

$$x_1 + x_2 + 2x_3 + x_4 + x_5 \leq 3.$$

For $p = (7, 10, 4, 4, 8)$, the strengthened relaxation gives

$$\max\{p^\top x : x \in P^{CG}\} = 25.$$

Since $x^* = (1, 1, 0, 0, 1) \in P_I$ and $p^\top x^* = 25$,

$$\max\{p^\top x : x \in P^{CG}\} = \max\{p^\top x : x \in P_I\}.$$

The CG cut therefore closes the root gap for this objective direction. This does not mean that this single cut gives a complete description of P_I .

Combinatorial cuts: cover inequalities

For a binary knapsack

$$\sum_i w_i x_i \leq b, \quad x_i \in \{0, 1\},$$

a set C is a cover if

$$\sum_{i \in C} w_i > b.$$

All items in C cannot be selected simultaneously.

Thus the cover inequality

$$\sum_{i \in C} x_i \leq |C| - 1$$

is valid.

A cover cut for the example

Take

$$C = \{1, 2, 4, 5\}.$$

Its total weight is

$$3 + 5 + 2 + 6 = 16 > 15.$$

Therefore

$$x_1 + x_2 + x_4 + x_5 \leq 3$$

is valid.

At the root LP solution,

$$x_1 + x_2 + x_4 + x_5 = 1 + 1 + 1 + \frac{5}{6} = 3.83 > 3.$$

The cover inequality is violated.

Effect of the cover cut

After adding

$$x_1 + x_2 + x_4 + x_5 \leq 3,$$

the relaxation bound becomes

$$\text{UB} = 25.33.$$

One optimal LP solution is

$$x_1 = x_2 = x_5 = 1, \quad x_3 = \frac{1}{12}, \quad x_4 = 0.$$

The relaxation is stronger, but still fractional.

$$25.33 > 25.$$

Extended cover inequality

Let C be a cover.

A standard extension is

$$E(C) = C \cup \left\{ j \notin C : w_j \geq \max_{i \in C} w_i \right\}.$$

Then

$$\sum_{i \in E(C)} x_i \leq |C| - 1$$

is valid.

The right-hand side remains $|C| - 1$, not $|E(C)| - 1$.

Extended cover for the example

For

$$C = \{1, 2, 4, 5\},$$

we have

$$\max_{i \in C} w_i = 6.$$

Since

$$w_3 = 12 \geq 6,$$

item 3 can be added to the extended cover.

$$x_1 + x_2 + x_3 + x_4 + x_5 \leq 3.$$

This cuts the fractional solution with $x_3 = 1/12$ and closes the root gap:

What the two cuts mean on the knapsack

Cut	Derivation	Interpretation
CG cut	Divide, combine, add bounds, then round because the variables are integer	Algebraic strengthening of the LP relaxation
Cover cut	Find a subset of items whose total weight exceeds the capacity	These items cannot all be selected together
Extended cover	Add items at least as heavy as the largest item in the cover	Replacing a cover item by a heavier one still violates capacity

The CG cut is generic: it sees integer variables and linear inequalities. The cover cuts are combinatorial: they see items, weights, and capacity.

Larger knapsack instance

We now use a larger instance to illustrate cuts generated away from the root.

$$\max 13x_1 + 10x_2 + 7x_3 + 6x_4 + 12x_5 + 11x_6 + 9x_7 + 8x_8$$

$$6x_1 + 5x_2 + 4x_3 + 4x_4 + 7x_5 + 6x_6 + 5x_7 + 5x_8 \leq 18.$$

$$x_i \in \{0, 1\}.$$

The root LP solution is

$$x = (1, 1, 0, 0, 0, 1, 0.2, 0), \quad \text{UB} = 35.8.$$

A non-root node

Branch on the fractional variable x_7 .

Consider the child node

$$x_7 = 1.$$

The LP relaxation at this node gives

$$x = (1, 1, 0, 0, 0, 1/3, 1, 0), \quad \text{UB} = 35.67.$$

This node is fractional because

$$x_6 = \frac{1}{3}.$$

The remaining capacity after fixing $x_7 = 1$ is

$$18 - 5 = 13.$$

A local CG cut at the node $x_7 = 1$

At the node $x_7 = 1$, the residual capacity is

$$6x_1 + 5x_2 + 4x_3 + 4x_4 + 7x_5 + 6x_6 + 5x_8 \leq 13.$$

Multiply by $1/5$:

$$\frac{6}{5}x_1 + x_2 + \frac{4}{5}x_3 + \frac{4}{5}x_4 + \frac{7}{5}x_5 + \frac{6}{5}x_6 + x_8 \leq \frac{13}{5}.$$

Rounding down the integer left-hand side gives the local CG cut

$$x_1 + x_2 + x_5 + x_6 + x_8 \leq 2.$$

The local CG cut is violated

At the node solution

$$x = (1, 1, 0, 0, 0, 1/3, 1, 0),$$

the left-hand side of the local CG cut is

$$x_1 + x_2 + x_5 + x_6 + x_8 = 1 + 1 + 0 + \frac{1}{3} + 0 = 2.33.$$

$$2.33 > 2.$$

The cut is generated at a non-root node.

It is local because it uses the restriction $x_7 = 1$.

A local cover cut at the same node

At the node $x_7 = 1$, the residual capacity is 13.

Take

$$C = \{1, 2, 6\}.$$

Its residual weight is

$$6 + 5 + 6 = 17 > 13.$$

Thus, inside this node,

$$x_1 + x_2 + x_6 \leq 2$$

is valid.

At the node LP solution,

$$x_1 + x_2 + x_6 = 1 + 1 + \frac{1}{3} = 2.33 > 2.$$

A local extended cover cut

For the local cover

$$C = \{1, 2, 6\},$$

we have

$$\max_{i \in C} w_i = 6.$$

Item 5 has weight 7, hence it can be added.

$$x_1 + x_2 + x_5 + x_6 \leq 2.$$

This is valid only in the subtree $x_7 = 1$.

It is not a global inequality for the original knapsack.

Branch-and-Cut: main idea

Bonobo provides the Branch-and-Bound tree machinery.

The cut loop is implemented inside node evaluation.

1. apply node restrictions,
2. solve relaxation,
3. separate violated cuts,
4. add cuts and re-solve,
5. return the final node bound.

For local cuts, each node stores the cuts valid in its subtree.

Global cuts versus local cuts

Global cuts are valid for the original problem.

$$\alpha^T x \leq \beta \quad \forall x \in X.$$

They can be kept permanently.

Local cuts are valid only after some branching decisions.

$$\alpha^T x \leq \beta \quad \forall x \in X \cap N.$$

They must be stored in the node and inherited by its descendants.

For pedagogical implementations, rebuilding the LP model at each node is often clearer than adding and deleting local constraints in a shared model.

Node type with local linear cuts

```
# Generic internal representation of any linear cut:
#   coeff' * x <= rhs
struct LinearCut
    coeff::Vector{Float64}
    rhs::Float64
    name::String
end

mutable struct CutNode <: BB.AbstractNode
    std::BB.BnBNodeInfo
    lbs::Vector{Float64}
    ubs::Vector{Float64}
    local_cuts::Vector{LinearCut}
    lp_objective::Float64
    lp_solution::Vector{Float64}
    integral::Bool
    cuts_added_at_node::Int
end
```

LinearCut is a user-defined structure, not a Bonobo object. All separated cuts, whether CG cuts or cover cuts, are stored in this common format.

Building the node relaxation

```
function build_node_model(node::CutNode)
    model = Model(HiGHS.Optimizer)
    set_silent(model)

    @variable(model, node.lbs[i] <= x[i=1:n] <= node.ubs[i])

    @constraint(model,
        sum(weight[i] * x[i] for i in 1:n) <= capacity
    )

    for cut in node.local_cuts
        @constraint(model,
            sum(cut.coeff[i] * x[i] for i in 1:n) <= cut.rhs
        )
    end

    @objective(model, Max,
        sum(profit[i] * x[i] for i in 1:n)
    )

    return model, x
end
```

Residual capacity at a node

For a knapsack node, fixed variables can be detected from the bounds.

```
function residual_capacity(node)
    fixed_one = [
        i for i in 1:n
        if node.lbs[i] == 1 && node.ubs[i] == 1
    ]

    return capacity - sum(weight[i] for i in fixed_one)
end

function free_indices(node)
    return [
        i for i in 1:n
        if !(node.lbs[i] == node.ubs[i])
    ]
end
```

Local cuts are generated on the residual subproblem.

CG separator for a knapsack node

For a multiplier $\theta > 0$, the residual CG cut is

$$\sum_{i \in F} \lfloor \theta w_i \rfloor x_i \leq \lfloor \theta b_N \rfloor.$$

```
function separate_cg_cuts(node, xval; theta_values = [1/2, 1/3, 1/4, 1/5, 1/6])
    bN = residual_capacity(node)
    F = free_indices(node)
    cuts = LinearCut[]

    for theta in theta_values
        coeff = zeros(n)
        for i in F
            coeff[i] = floor(theta * weight[i])
        end
        rhs = floor(theta * bN)

        if sum(coeff[i] * xval[i] for i in 1:n) > rhs + 1e-6
            push!(cuts, LinearCut(coeff, rhs, "CG theta=$(theta)"))
        end
    end

    return cuts
end
```

CG Branch-and-Cut node evaluation: solve

```
function BB.evaluate_node!(tree, node::CutNode)
    node.cuts_added_at_node = 0

    while true
        model, x = build_node_model(node)
        optimize!(model)

        if termination_status(model) != MOI.OPTIMAL
            return NaN, NaN
        end

        node.lp_objective = objective_value(model)
        node.lp_solution = value.(x)

        # separation on the next slide
    end
end
```

The node model is rebuilt after each new local cut.

CG Branch-and-Cut node evaluation: separate

```
new_cuts = separate_cg_cuts(node, node.lp_solution)

if isempty(new_cuts)
    break
end

append!(node.local_cuts, new_cuts)
node.cuts_added_at_node += length(new_cuts)
```

After appending the cuts, the relaxation is rebuilt and re-solved.

At the node $x_7 = 1$, this separator can generate

$$x_1 + x_2 + x_5 + x_6 + x_8 \leq 2.$$

CG Branch-and-Cut node evaluation: return

```
node.integral = all(  
    BB.is_approx_feasible.(tree, node.lp_solution)  
)  
  
if node.integral  
    return node.lp_objective, node.lp_objective  
else  
    return node.lp_objective, NaN  
end
```

The returned bound is the value after all separated CG cuts have been added.

Children inherit local cuts

When a node is branched, its children inherit its local cuts.

```
push!(children, (  
    lbs = child_lbs,  
    ubs = child_ubs,  
    local_cuts = copy(node.local_cuts),  
    lp_objective = NaN,  
    lp_solution = fill(NaN, n),  
    integral = false,  
    cuts_added_at_node = 0,  
))
```

A cut valid in a node remains valid in every descendant of that node.

Cover-separation logic

At a node, fixed-one variables reduce the residual capacity.

$$b_N = b - \sum_{i: x_i=1 \text{ fixed}} w_i.$$

A local cover is a set C of free variables satisfying

$$\sum_{i \in C} w_i > b_N.$$

The corresponding local cut is

$$\sum_{i \in C} x_i \leq |C| - 1.$$

Cover cuts as linear cuts

A cover cut is still a linear inequality. We store it directly as a `LinearCut`, using coefficients equal to one on the cover set.

```
function cover_to_linear_cut(C, rhs; name = "cover")
    coeff = zeros(n)
    for i in C
        coeff[i] = 1.0
    end
    return LinearCut(coeff, rhs, name)
end
```

For example, $C = \{1, 3, 4\}$ and $\text{rhs} = 2$ gives

$$x_1 + x_3 + x_4 \leq 2.$$

A simple cover separator

A minimal separator can enumerate small covers and return `LinearCut` objects directly.

```
function separate_cover_cuts(node, xval; max_size = 4)
    bN = residual_capacity(node)
    F = free_indices(node)
    cuts = LinearCut[]

    for k in 2:max_size
        for C in combinations(F, k)
            if sum(weight[i] for i in C) > bN
                rhs = k - 1
                lhs = sum(xval[i] for i in C)

                if lhs > rhs + 1e-6
                    push!(cuts,
                        cover_to_linear_cut(C, rhs; name = "cover"))
                end
            end
        end
    end

    return cuts
end
```

This is a limited separator: it only tests covers of small cardinality. Exact separation is obtained by solving an auxiliary knapsack problem.

Separation of cover inequalities

Consider the residual knapsack at a node:

$$\sum_{i \in F} w_i x_i \leq b_N, \quad x_i \in \{0, 1\}.$$

Let $\bar{x}$ be the current LP solution. A cover $C \subseteq F$ satisfies

$$\sum_{i \in C} w_i > b_N.$$

The cover inequality is violated by $\bar{x}$ when

$$\sum_{i \in C} \bar{x}_i > |C| - 1 \iff \sum_{i \in C} (1 - \bar{x}_i) < 1.$$

Therefore separation searches for an overweight set with minimum cost $1 - \bar{x}_i$.

Exact separation as an auxiliary knapsack

Introduce binary variables

$$y_i = 1 \iff i \in C.$$

For integer weights and integer residual capacity, the strict cover condition becomes

$$\sum_{i \in F} w_i y_i \geq b_N + 1.$$

The exact separation problem is

$$\begin{aligned} \gamma^* = \min \quad & \sum_{i \in F} (1 - \bar{x}_i) y_i \\ \text{s.t.} \quad & \sum_{i \in F} w_i y_i \geq b_N + 1, \\ & y_i \in \{0, 1\}. \end{aligned}$$

Then $\gamma^* < 1$ if and only if a violated cover inequality exists.

Violation value and standard knapsack form

If y^* is optimal and

$$C = \{i \in F : y_i^* = 1\},$$

then the violated inequality is

$$\sum_{i \in C} x_i \leq |C| - 1.$$

Its violation at $\bar{x}$ is

$$\sum_{i \in C} \bar{x}_i - (|C| - 1) = 1 - \sum_{i \in C} (1 - \bar{x}_i) = 1 - \gamma^*.$$

By setting $z_i = 1 - y_i$, the same problem can be written as a standard maximum knapsack problem. Hence exact separation of basic cover cuts is itself an optimization problem.

Exact separation on the reference knapsack

At the root LP solution,

$$\bar{x} = (1, 1, 0, 1, 5/6), \quad 1 - \bar{x} = (0, 0, 1, 0, 1/6).$$

The exact separation problem is

$$\begin{aligned} \min \quad & y_3 + \frac{1}{6}y_5 \\ \text{s.t.} \quad & 3y_1 + 5y_2 + 12y_3 + 2y_4 + 6y_5 \geq 16, \\ & y_i \in \{0, 1\}. \end{aligned}$$

One optimal solution is

$$y = (1, 1, 0, 1, 1), \quad C = \{1, 2, 4, 5\}.$$

Therefore $x_1 + x_2 + x_4 + x_5 \leq 3$ is violated by $5/6$.

Exact cover separator in JuMP

```
function separate_cover_cut_exact(node, xval; eps = 1e-6)
    bN = residual_capacity(node)
    F = free_indices(node)

    model = Model(HiGHS.Optimizer)
    set_silent(model)

    @variable(model, y[i in F], Bin)

    @constraint(model,
        sum(weight[i] * y[i] for i in F) >= bN + 1)

    @objective(model, Min,
        sum((1.0 - xval[i]) * y[i] for i in F))

    optimize!(model)
    termination_status(model) == MOI.OPTIMAL || return LinearCut[]

    if objective_value(model) < 1.0 - eps
        C = [i for i in F if value(y[i]) > 0.5]
        rhs = length(C) - 1
        return [cover_to_linear_cut(C, rhs; name = "exact cover")]
    end

    return LinearCut[]
end
```


Exact versus heuristic separation

Heuristic enumeration	Exact separation
Tests only covers up to a prescribed size	Searches over all covers
Fast and easy to implement	Requires solving an auxiliary IP
May miss a violated cover	Finds a violated cover whenever one exists
Useful inside Branch-and-Cut when time is limited	Useful to define the separation problem precisely

The auxiliary problem above separates basic cover inequalities. Separating all lifted or extended-cover inequalities is a stronger and different task.

Extending a cover

```
function extend_cover(node, C)
  F = free_indices(node)
  threshold = maximum(weight[i] for i in C)

  E = copy(C)

  for j in F
    if !(j in C) && weight[j] >= threshold
      push!(E, j)
    end
  end

  return sort(unique(E))
end
```

The right-hand side stays $|C| - 1$, not $|E| - 1$.

Cover and extended-cover separator

The separator returns only LinearCut objects.

```
function separate_extended_cover_cuts(node, xval; max_size = 4)
    bN = residual_capacity(node)
    F = free_indices(node)
    cuts = LinearCut[]

    for k in 2:max_size, C0 in combinations(F, k)
        C = collect(C0)
        sum(weight[i] for i in C) > bN || continue

        rhs = length(C) - 1
        E = extend_cover(node, C)

        if sum(xval[i] for i in E) > rhs + 1e-6
            push!(cuts, cover_to_linear_cut(E, rhs;
                name = "extended cover"))
        elseif sum(xval[i] for i in C) > rhs + 1e-6
            push!(cuts, cover_to_linear_cut(C, rhs; name = "cover"))
        end
    end

    return cuts
end
```

Node cut on the larger example

At the node $x_7 = 1$, the residual capacity is 13.

The set

$$C = \{1, 2, 6\}$$

is a local cover:

$$6 + 5 + 6 = 17 > 13.$$

It gives

$$x_1 + x_2 + x_6 \leq 2.$$

Since $w_5 = 7 \geq 6$, the extended cover is

$$x_1 + x_2 + x_5 + x_6 \leq 2.$$

Violation at the non-root node

The LP solution at the node $x_7 = 1$ is

$$x = (1, 1, 0, 0, 0, 1/3, 1, 0).$$

For the cover inequality,

$$x_1 + x_2 + x_6 = 2.33 > 2.$$

For the extended cover inequality,

$$x_1 + x_2 + x_5 + x_6 = 2.33 > 2.$$

A local cover cut is generated inside the B&B tree, not only at the root.

Cover Branch-and-Cut node evaluation

The code structure is the same as for CG cuts.

```
function BB.evaluate_node!(tree, node::CutNode)
    node.cuts_added_at_node = 0

    while true
        model, x = build_node_model(node)
        optimize!(model)

        if termination_status(model) != MOI.OPTIMAL
            return NaN, NaN
        end

        node.lp_objective = objective_value(model)
        node.lp_solution = value.(x)

        new_cuts = separate_extended_cover_cuts(node, node.lp_solution)

        isempty(new_cuts) && break

        append!(node.local_cuts, new_cuts)
        node.cuts_added_at_node += length(new_cuts)
    end

    # integrality test and return as before
end
```

Experimental protocol

We compare the two pedagogical Branch-and-Cut variants on the larger instance.

Item	Choice in the experiment
Instance	8-item knapsack from the previous slides
Root LP	$UB = 35.8$
Optimal integer value	$z^* = 35$
Branching rule	first fractional variable
Tree search	depth-first search
CG separator	multipliers $1/2, 1/3, 1/4, 1/5, 1/6$
Cover separator	covers of size at most 4, with simple extension
Timing	median CPU time over 30 runs

Cuts generated at the root are counted as global cuts. Cuts generated below the root are counted as local cuts.

Numerical comparison

Both variants return the same optimal solution:

$$x = (1, 1, 0, 0, 1, 0, 0, 0), \quad z^* = 35.$$

Measure	CG cuts	Cover / extended-cover cuts
CPU time, median over 30 runs	23.6 ms	2.7 ms
B&B nodes evaluated	15	1
LP relaxations solved	22	2
Global cuts generated at root	1	6
Local cuts generated below root	13	0
Total cuts generated	14	6
Maximum tree depth	5	0

The values depend on the implementation and on the machine. They are reported only to compare the two pedagogical algorithms on this instance.

What the comparison shows

The comparison highlights two distinct behaviours on this instance.

- CG cuts are generic and cheap to describe, but the chosen multiplier scan produces relatively weak cuts on this example.
- The CG version therefore needs branching, and many cuts are generated locally inside the tree.
- Cover and extended-cover cuts exploit the knapsack structure directly.
- On this instance, the cover version closes the gap at the root, so no branching is needed.

The point is not that cover cuts are uniformly superior. Rather, a problem-specific separator may substantially reduce the search tree.

Where the two algorithms differ

Component	CG Branch-and-Cut	Cover Branch-and-Cut
cut source	rounding	knapsack structure
separator	multiplier scan	cover search
node data	local linear cuts	local linear cuts
cut validity	local or global	local or global
strength	generic	problem-specific

The surrounding Bonobo code is unchanged; only the separator is replaced.

Local cuts in a shared-model implementation

If the same JuMP model is reused for all nodes, global cuts are easy:

add once and keep permanently.

Local cuts are more delicate:

valid only in a subtree.

Possible implementations:

- rebuild the node LP relaxation from scratch;
- add local cuts temporarily and delete them after node evaluation;
- maintain cut activation constraints;
- store local cuts in each node and pass them to descendants.

Rebuilding the relaxation is the clearest method for a teaching notebook.

Generic Branch-and-Cut template in Bonobo

1. Select a node N .
2. Build the relaxation of N .
3. Add inherited local cuts.
4. Solve the relaxation.
5. Separate violated cuts.
6. If cuts are found, store them in N and repeat.
7. Return the final node bound.
8. Let Bonobo fathom, update incumbent, or branch.
9. When branching, copy local cuts to children.

Logging useful information

For a Branch-and-Cut notebook, it is useful to record:

- node identifier and parent identifier;
- branching restriction;
- final LP bound after cuts;
- number of cuts added at the node;
- incumbent value after processing the node;
- fathoming reason.

The tree visualization can then distinguish:

pure branching nodes

cut-generation nodes

fathomed nodes.

When the complete formulation is too large

Many integer programs have a compact set of variables and an exponential family of constraints.

A standard formulation has the form

$$\min\{c^T x : Ax \leq b, Hx \leq h, x \in \mathbb{Z}^n\},$$

where $Ax \leq b$ is small, but $Hx \leq h$ contains exponentially many valid inequalities.

Instead of writing all constraints at once, one starts from the small relaxation

$$Ax \leq b,$$

and generates missing constraints only when they are needed.

This is the principle of lazy-constraint generation.

TSP: a compact relaxation is not enough

For the symmetric traveling salesman problem, let

$$x_e = 1 \iff e \text{ belongs to the tour.}$$

The degree equations are compact:

$$\sum_{e \in \delta(v)} x_e = 2, \quad v \in V.$$

But they only force every vertex to have degree two.

They can produce several disjoint cycles instead of one Hamiltonian cycle.

To forbid subtours, one needs the subtour elimination constraints

$$\sum_{e \in E(S)} x_e \leq |S| - 1, \quad \emptyset \neq S \subsetneq V.$$

There are exponentially many such subsets S .

Lazy cuts: constraints generated at integer candidates

A lazy cut is a valid constraint that is part of the intended integer formulation, but is not inserted initially.

It is separated when the solver finds a candidate integer solution.

candidate integer solution $\implies$ check missing constraints

violated constraint found $\implies$ add it lazily and reject the candidate.

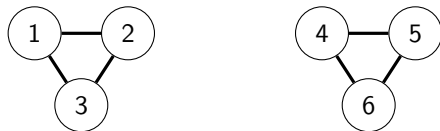
For TSP, the candidate solution may satisfy all degree equations but contain two or more subtours. Each subtour gives a violated subtour elimination constraint.

If no violated lazy constraint is found, the candidate is feasible for the full exponential formulation.

Example: two subtours in a TSP candidate

Suppose the degree equations return two disjoint cycles:

$$1 - 2 - 3 - 1, \quad 4 - 5 - 6 - 4.$$



For $S = \{1, 2, 3\}$, the subtour cut is

$$x_{12} + x_{13} + x_{23} \leq 2.$$

For $S = \{4, 5, 6\}$, the subtour cut is

$$x_{45} + x_{46} + x_{56} \leq 2.$$

These cuts remove the incumbent candidate, but keep all Hamiltonian tours.

Lazy separation loop for the TSP

1. Solve the MIP with degree equations and $x_e \in \{0, 1\}$.
2. When an integer candidate $\bar{x}$ appears, build the graph $G(\bar{x})$.
3. Find the connected components of $G(\bar{x})$.
4. If there is one component, $\bar{x}$ is a Hamiltonian tour.
5. If there are several components $S_1, \dots, S_k$, add

$$\sum_{e \in E(S_r)} x_e \leq |S_r| - 1, \quad r = 1, \dots, k.$$

6. Continue the branch-and-bound search.

The solver never needs to enumerate all subsets $S \subset V$ in advance.

Lazy cuts versus ordinary cutting planes

	Ordinary cutting plane	Lazy cut
Main role	Strengthen the LP relaxation	Complete a formulation with missing constraints
Separated at	Usually fractional LP solutions	Usually integer candidate solutions
Can remove an incumbent candidate?	No, if it is feasible for the full model	Yes, if it violates an omitted constraint
Typical examples	CG cuts, MIR cuts, cover cuts, clique cuts	TSP subtour cuts, connectivity cuts, capacity cuts in vehicle routing, cutset constraints in network design

In practice, Branch-and-Cut may use both mechanisms:

user cuts for bounds + lazy cuts for correctness.

Other problems with useful lazy cuts

Lazy cuts are natural when feasibility has a global combinatorial condition.

Problem	Typical lazy constraints
TSP	subtour elimination constraints
Vehicle routing	capacity cuts and route-elimination cuts
Steiner tree	connectivity cuts separating terminals from the root
Survivable network design	cutset inequalities enforcing required connectivity
Benders decomposition	feasibility or optimality cuts generated from a sub-problem

The common scheme is to solve a smaller master model, inspect the candidate solution, and add only constraints that are actually violated.

Summary

Cutting planes strengthen relaxations.

Branch-and-Cut places the cutting-plane loop inside Branch-and-Bound nodes.

For the small knapsack:

one CG cut already reaches the integer-hull bound for the objective.

For the larger knapsack:

local cuts can be generated after branching.

In Bonobo:

tree management is provided by Bonobo; cut separation is implemented by the user.

Richard Lusby and Thomas Stidsen, *Mathematical Programming with Julia*, Section 5.2, “Solution algorithm: Branch & Bound”.

Bonobo.jl documentation:

<https://wikunia.github.io/Bonobo.jl/>

JuMP documentation:

<https://jump.dev/JuMP.jl/stable/>

For background on cover inequalities and Chvatal–Gomory cuts, see standard textbooks on integer programming and polyhedral combinatorics.